

Code:

```
#include <iostream>
#include <cstring>
using namespace std;

// Forward declarations of classes
class Project;
class Developer;
class DevProj;

// -----
// Class: Project
// Represents a project assigned to a developer
// -----
class Project {
    int Project_ID;      // Unique ID for the project
    int Duration;        // Duration in months
    int Cost;            // Cost of the project
    char Status[20];     // Status of the project (e.g., Active, Completed)
    Project *next;       // Pointer to the next project (for circular linked
list)

    friend class Developer;
    friend class DevProj;

public:
    // Parameterized constructor
    Project(int id, int dur, int cst, const char* stat) {
        Project_ID = id;
        Duration = dur;
        Cost = cst;
        strcpy(Status, stat);
        next = nullptr;
    }

    // Print project details
    void Print() {
        cout << "\n-----\n";
        cout << "          PROJECT DETAILS          \n";
        cout << "-----\n";
    }
};
```

```

        cout << "Project ID      : " << Project_ID << endl;
        cout << "Duration (months): " << Duration << endl;
        cout << "Cost (USD)       : " << Cost << endl;
        cout << "Status          : " << Status << endl;
    }

    // Return project ID
    int getID() { return Project_ID; }
};

// -----
// Class: Developer
// Represents a developer with their list of projects
// Doubly linked list of developers
// Each developer has a circular linked list of projects
// -----
class Developer {
    int Dev_ID;           // Unique ID for developer
    char Name[30];        // Developer name
    char Speciality[20];  // Developer's specialization (e.g., Web,
Mobile)
    int Project_Count;    // Total number of assigned projects

    Developer *next;      // Pointer to next developer
    Developer *prev;      // Pointer to previous developer

    Project *head;        // Head of circular linked list of projects
    Project *tail;        // Tail of project list

    friend class DevProj;

public:
    // Parameterized constructor
    Developer(int id, const char* name, const char* spec) {
        Dev_ID = id;
        strcpy(Name, name);
        strcpy(Speciality, spec);
        Project_Count = 0;
        next = nullptr;
        prev = nullptr;
        head = nullptr;
        tail = nullptr;
    }

    // Add a project to the developer's circular linked list
    void AddProject(Project *proj) {
        if (proj == nullptr) return;

```

```

        // If no projects yet, make this the first one
        if (head == nullptr) {
            head = proj;
            tail = proj;
            proj->next = proj; // circular link
        } else {
            // Insert at the beginning of circular list
            proj->next = head;
            tail->next = proj;
            head = proj;
        }
        Project_Count++;
    }

    // Print developer details
    void Print() {
        cout << "\n===== \n";
        cout << "          DEVELOPER PROFILE          \n";
        cout << "===== \n";
        cout << "Developer ID   : " << Dev_ID << endl;
        cout << "Name           : " << Name << endl;
        cout << "Specialization : " << Speciality << endl;
        cout << "Total Projects : " << Project_Count << endl;
    }

    // Return developer ID
    int getID() { return Dev_ID; }

    // Return total projects
    int getProjectCount() { return Project_Count; }

    // Return head of project list
    Project* getHead() { return head; }
};

// -----
// Class: DevProj
// Main controller class that manages developers and projects
// Handles all user operations like search, add, view, navigate
// -----
class DevProj {
    Developer *head;           // Start of developer doubly linked list
    Developer *tail;           // End of developer list
    Developer *current;        // Pointer to current developer being viewed
    Project *currentProject;    // Pointer to current project being viewed

public:
    // Constructor initializes pointers

```

```

DevProj() {
    head = nullptr;
    tail = nullptr;
    current = nullptr;
    currentProject = nullptr;
}

// Add a new developer node to the doubly linked list
void AddDeveloper(Developer *dev) {
    if (dev == nullptr) return;

    if (head == nullptr) {
        head = dev;
        tail = dev;
    } else {
        dev->next = head;
        head->prev = dev;
        head = dev;
    }
    cout << "\nDeveloper record added successfully.\n";
}

// Assign a project to a specific developer
void AddProject(Developer *dev, Project *proj) {
    if (dev != nullptr && proj != nullptr) {
        dev->AddProject(proj);
        cout << "\nProject assigned successfully.\n";
    }
}

// Search developer by ID
Developer* SearchDeveloper(int id) {
    Developer *temp = head;
    while (temp != nullptr) {
        if (temp->Dev_ID == id) {
            return temp;
        }
        temp = temp->next;
    }
    return nullptr;
}

// Search project by ID in all developers
bool SearchProject(int projID) {
    Developer *dev = head;
    while (dev != nullptr) {
        if (dev->head != nullptr) {
            Project *proj = dev->head;

```

```

        Project *start = proj;

        // Traverse circular list
        do {
            if (proj->Project_ID == projID) {
                cout << "\n-----\n";
                cout << "        PROJECT SEARCH RESULT        \n";
                cout << "-----\n";
                dev->Print();
                proj->Print();
                return true;
            }
            proj = proj->next;
        } while (proj != start);
    }
    dev = dev->next;
}

cout << "\nNo project found with the given ID.\n";
return false;
}

// View all developers (starts from head)
void ViewDevelopers() {
    if (head == nullptr) {
        cout << "\nNo developers are currently registered in the
system.\n";
        return;
    }
    current = head;
    current->Print();
}

// Move to next developer
void ShowNextDeveloper() {
    if (current == nullptr) {
        cout << "\nPlease view the developer list first using 'v'. \n";
        return;
    }

    if (current->next == nullptr) {
        cout << "\nYou are already viewing the last developer record.\n";
        return;
    }

    current = current->next;
    current->Print();
    currentProject = nullptr;
}
}

```

```

// Move to previous developer
void ShowPreviousDeveloper() {
    if (current == nullptr) {
        cout << "\nPlease view the developer list first using 'v'.\n";
        return;
    }

    if (current->prev == nullptr) {
        cout << "\nYou are already viewing the first developer record.\n";
        return;
    }

    current = current->prev;
    current->Print();
    currentProject = nullptr;
}

// Show developer details and their first project
void ShowDeveloperWithProjects(int devID) {
    Developer *dev = SearchDeveloper(devID);
    if (dev == nullptr) {
        cout << "\nNo developer found with the given ID.\n";
        return;
    }

    current = dev;
    dev->Print();

    // Show the first project if available
    if (dev->head == nullptr) {
        cout << "\nThis developer currently has no assigned projects.\n";
        currentProject = nullptr;
    } else {
        currentProject = dev->head;
        currentProject->Print();
    }
}

// Show next project in the current developer's circular list
void ShowNextProject() {
    if (current == nullptr) {
        cout << "\nPlease search for a developer first using 's'.\n";
        return;
    }

    if (current->head == nullptr) {
        cout << "\nThe selected developer has no projects.\n";
    }
}

```

```

        return;
    }

    if (currentProject == nullptr) {
        currentProject = current->head;
    } else {
        currentProject = currentProject->next;
    }

    currentProject->Print();
}

// Destructor - deallocates all developer and project nodes
~DevProj() {
    Developer *dev = head;
    while (dev != nullptr) {
        if (dev->head != nullptr) {
            Project *proj = dev->head;
            Project *start = proj;
            do {
                Project *toDelete = proj;
                proj = proj->next;
                delete toDelete;
            } while (proj != start && proj != dev->head);
        }
        Developer *toDelete = dev;
        dev = dev->next;
        delete toDelete;
    }
}

};

// -----
// Function: displayMenu
// Displays all options available to the user
// -----
void displayMenu() {
    cout << "\n===== \n";
    cout << "          DEVPROJ MANAGEMENT CONSOLE          \n";
    cout << "===== \n";
    cout << "[D] Add a Developer\n";
    cout << "[P] Add a Project\n";
    cout << "[S] Search Developer\n";
    cout << "[T] Search Project\n";
    cout << "[V] View Developer List\n";
    cout << "[F] Next Developer\n";
    cout << "[R] Previous Developer\n";
    cout << "[N] Next Project\n";
}

```

```

    cout << "[E] Exit Program\n";
    cout << "-----\n";
    cout << "Enter your choice: ";
}

// -----
// Main Function
// Handles menu-driven user interaction
// -----
int main() {
    DevProj platform; // Main object controlling developers & projects
    char choice;

    cout << "\n===== \n";
    cout << "      WELCOME TO THE DEVPROJ MANAGEMENT TOOL  \n";
    cout << "===== \n";

    while (true) {
        displayMenu();
        cin >> choice;
        cin.ignore();

        switch (choice) {
            // Add developer
            case 'd': case 'D': {
                int id;
                char name[30], spec[20];
                cout << "\nEnter Developer ID: ";
                cin >> id;
                cin.ignore();
                cout << "Enter Developer Name: ";
                cin.getline(name, 30);
                cout << "Enter Specialization (Web/Mobile/Desktop): ";
                cin.getline(spec, 20);
                platform.AddDeveloper(new Developer(id, name, spec));
                break;
            }

            // Add project to a developer
            case 'p': case 'P': {
                int devID, projID, duration, cost;
                char status[20];
                cout << "\nEnter Developer ID: ";
                cin >> devID;

                Developer *dev = platform.SearchDeveloper(devID);
                if (dev == nullptr) {
                    cout << "\nDeveloper not found in the system.\n";
                }
            }
        }
    }
}

```



```

        break;
    }
    cout << "Enter Project ID: ";
    cin >> projID;
    cout << "Enter Duration (months): ";
    cin >> duration;
    cout << "Enter Project Cost: ";
    cin >> cost;
    cin.ignore();
    cout << "Enter Project Status: ";
    cin.getline(status, 20);

    platform.AddProject(dev, new Project(projID, duration, cost,
status));

    break;
}

// Search developer
case 's': case 'S': {
    int id;
    cout << "\nEnter Developer ID to search: ";
    cin >> id;
    platform.ShowDeveloperWithProjects(id);
    break;
}

// Search project
case 't': case 'T': {
    int projID;
    cout << "\nEnter Project ID to search: ";
    cin >> projID;
    platform.SearchProject(projID);
    break;
}

// View developer list
case 'v': case 'V': {
    platform.ViewDevelopers();
    break;
}

// Move to next developer
case 'f': case 'F': {
    platform.ShowNextDeveloper();
    break;
}

// Move to previous developer

```

```

        case 'r': case 'R': {
            platform.ShowPreviousDeveloper();
            break;
        }

        // Move to next project
        case 'n': case 'N': {
            platform.ShowNextProject();
            break;
        }

        // Exit program
        case 'e': case 'E': {
            cout << "\nExiting the DevProj Management Tool. Goodbye!\n";
            return 0;
        }

        // Invalid input handling
        default:
            cout << "\nInvalid input. Please enter a valid option.\n";
    }
}

return 0;
}

```

Output:

Adding Developer1:

```

=====
WELCOME TO THE DEVPROJ MANAGEMENT TOOL
=====

=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
=====
Enter your choice: d

Enter Developer ID: 1
Enter Developer Name: moiz
Enter Specialization (Web/Mobile/Desktop): Web

Developer record added successfully.

```

Adding Developer2:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
-----
Enter your choice: d

Enter Developer ID: 2
Enter Developer Name: abdul
Enter Specialization (Web/Mobile/Desktop): Mobile

Developer record added successfully.
```

Adding Project:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
-----
Enter your choice: p

Enter Developer ID: 1
Enter Project ID: 11
Enter Duration (months): 2
Enter Project Cost: 10000o000000
Enter Project Status:
Project assigned successfully.
```

Search Project:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
-----
Enter your choice: t

Enter Project ID to search: 11

-----
PROJECT SEARCH RESULT
-----

=====
DEVELOPER PROFILE
=====
Developer ID   : 1
Name          : moiz
Specialization : Web
Total Projects : 1

-----
PROJECT DETAILS
-----
Project ID      : 11
Duration (months): 2
Cost (USD)      : 10000
Status         : 000000
```

Search Developer:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
-----
Enter your choice: s

Enter Developer ID to search: 1

=====
DEVELOPER PROFILE
=====
Developer ID   : 1
Name          : moiz
Specialization : Web
Total Projects : 1

-----
PROJECT DETAILS
-----
Project ID      : 11
Duration (months): 2
Cost (USD)      : 10000
Status         : 000000
```

View Developer :

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
=====
Enter your choice: v

=====
DEVELOPER PROFILE
=====
Developer ID   : 2
Name           : abdul
Specialization : Mobile
Total Projects : 0
```

Next Developer:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
=====
Enter your choice: f

=====
DEVELOPER PROFILE
=====
Developer ID   : 1
Name           : moiz
Specialization : Web
Total Projects : 1
```

Previous Developer:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
-----
Enter your choice: r

=====
DEVELOPER PROFILE
=====
Developer ID   : 2
Name           : abdul
Specialization : Mobile
Total Projects : 0
```

Next project:

```
=====
DEVPROJ MANAGEMENT CONSOLE
=====
[D] Add a Developer
[P] Add a Project
[S] Search Developer
[T] Search Project
[V] View Developer List
[F] Next Developer
[R] Previous Developer
[N] Next Project
[E] Exit Program
-----
Enter your choice: n

The selected developer has no projects.
```

Exit:

=====

DEVPROJ MANAGEMENT CONSOLE

=====

- [D] Add a Developer
- [P] Add a Project
- [S] Search Developer
- [T] Search Project
- [V] View Developer List
- [F] Next Developer
- [R] Previous Developer
- [N] Next Project
- [E] Exit Program

Enter your choice: e

Exiting the DevProj Management Tool. Goodbye!

PS C:\Users\Abdul Moiz\Desktop\code\test> █